

Arm Cortex M3 based SoC

快速搭建方法



内容提要

- Arm Processors
- Arm DesignStart
- Arm Cortex M3
- AMBA3 AHB-Lite总线协议
- Arm 杯破题与赛程规划
- 后续培训预告



Arm Processors

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用



Arm Processors

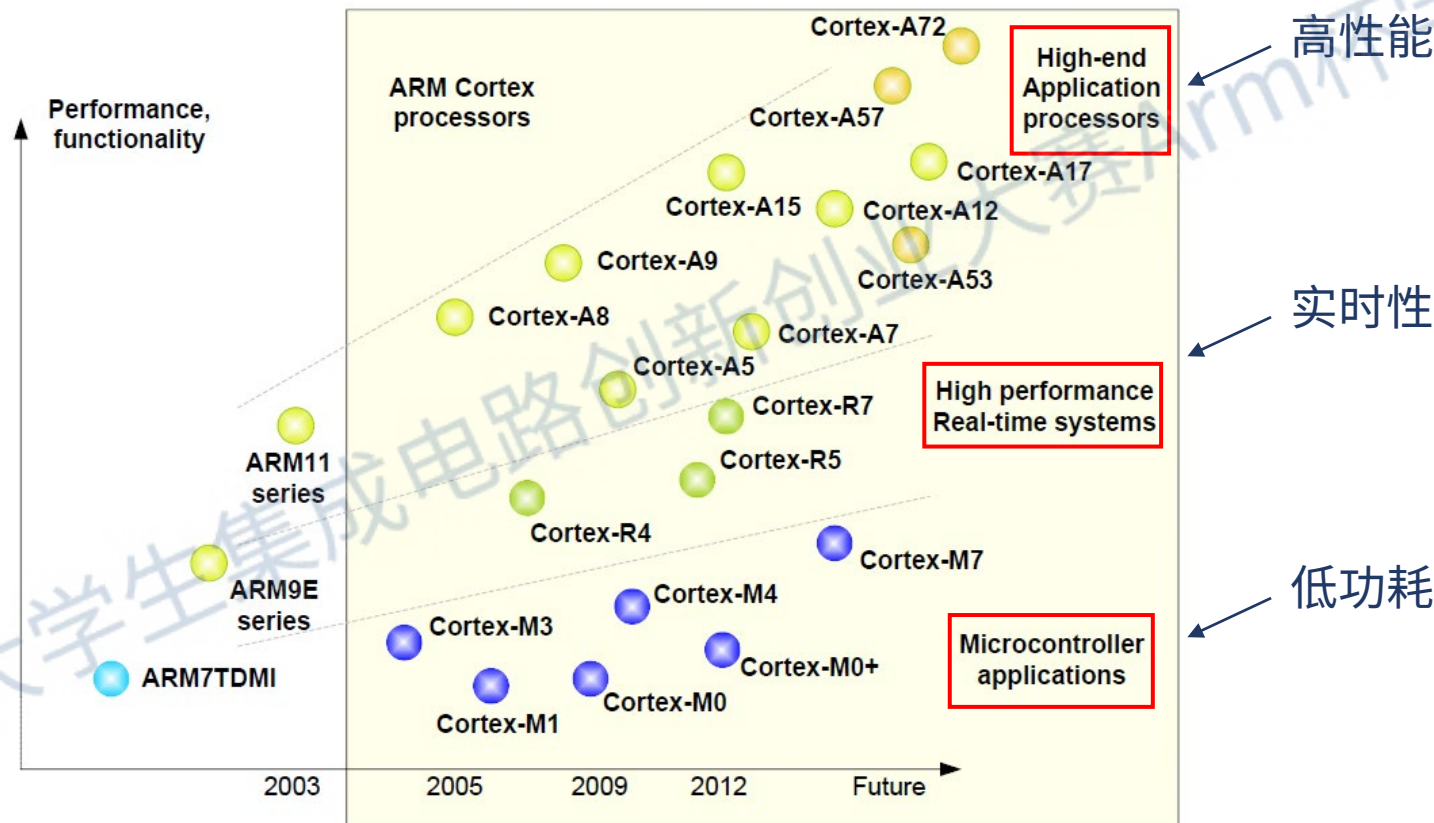


Figure 1.3

Overview of the ARM processor family.

《The Definitive Guide to Arm Cortex M0 and Cortex M0+ Processors》 Joseph Yiu



Arm Processors

- Cortex-A 处理器
- 处理高端操作系统(iOS, Android, Linux, Windows)
- 较长流水线, 可以运行在相对较高的时钟频率下(超过1GHz)
- 具有高级OS的虚拟存储器寻址操作所需的存储器管理单元(MMU)
- 可选的增强Java支持和名为TrustZone的安全程序执行环境
- 缺点: 无法对硬件事件作出快速反应(即实时性需求)

Arm Processors

- Cortex-R 处理器
- 实时、高性能，尤其适合数据分析
- 可以运行在较高时钟频率下(500MHz - 1GHz)
- 可以对硬件事件作出快速反应
- 缺点：设计复杂、功耗高

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用

Arm Processors

- Cortex-M 处理器
- 较短流水线
- 功耗低
- 工作时钟频率较低
- 中断管理功能强大且易于使用(NVIC)

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用

Arm Cortex M系列

- M0: 最小的Arm处理器，功耗低但能效效率高
- M0+: 大小和M0类似，但系统级调试特性更多
- M1: 和M0类似，但具有专用于FPGA的存储器系统特性
- M3: 性能比M0高很多，哈佛总线架构
- M4: 支持M3所有特性，且可以添加浮点单元(FPU)
- M7: 高性能处理器，同时支持单精度和双精度浮点运算
- 选择Arm DesignStart计划开放的Cortex M3来搭建SoC

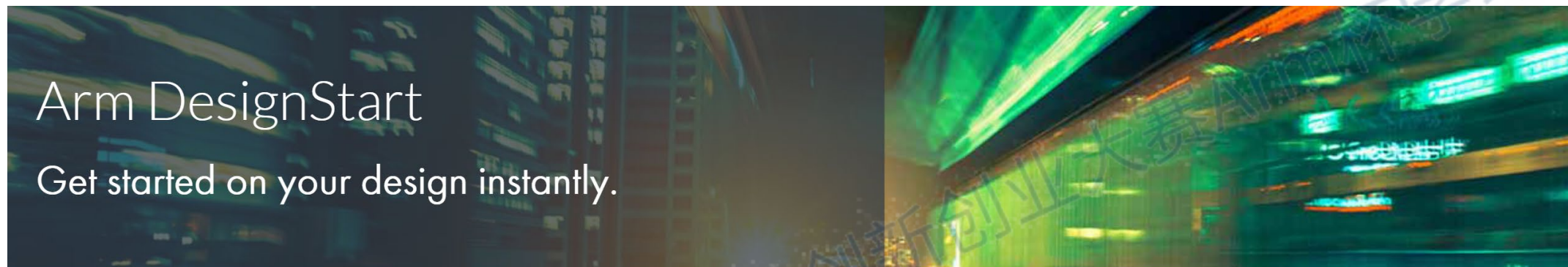


硬木课堂
www.emooc.cc

Arm DesignStart

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用

Arm DesignStart



What is DesignStart?

Arm DesignStart provides fast access to a select mix of Arm IP, including Cortex-M0, Cortex-M3 and Cortex-A5 processors and supporting IP, software and resources for custom silicon designs,. The program also provides Cortex-M1 and Cortex-M3 soft CPU IP, software and resources for FPGA designs.

Explore the DesignStart options below to find the route that best meets your project requirements.

■ <https://developer.Arm.com/ip-products/designstart>



DesignStart Eval版

■ 网表级verilog代码

■ 免费申请

■ 可读性差

DesignStart Eval

Design and prototype your SoC for evaluating, research and teaching

Cost: Free

Access: Instant access

License: Simple click-through end user license agreement



[Access Technical Resources](#)

DesignStart FPGA版

■ IP核

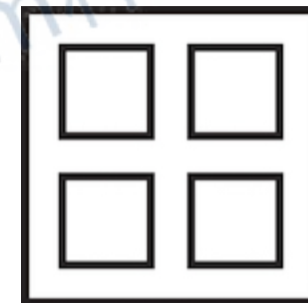
DesignStart FPGA

Instant access to Cortex-M IP, optimised for FPGA

Cost: Free

Access: Instant access

License: Simple click-through end user license agreement



■ 免费申请

■ 针对FPGA优化

Access Technical Resources



DesignStart Pro版

■ RTL级verilog代码

DesignStart Pro

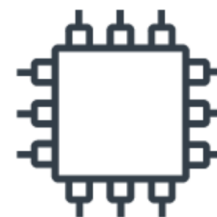
Start developing your commercial SoC

Cortex-M0 and Cortex-M3: No license fee and a success-based royalty model

Cortex-A5: Low-cost access fee with up to 3 years' design support. Minimal tape-out fee and a success-based royalty

Access: Requires signature

License: License agreement



■ 需要License

■ 不便于学习

Access Technical Resources

DesignStart Physical&University版

■ 物理实现IP

DesignStart Physical IP

Speed up SoC implementation

Cost: \$0 license fee and success-based royalty model

Access: Requires signature

License: License agreement

[Learn more](#)



■ 大学计划

DesignStart for University

SoC design and production for teaching and research

Cost: \$0 license fee (up to 1000 units for internal use)

Access: On application

License: Simple EULA

[Learn more](#)



Arm DesignStart

通过Arm DesignStart计划，我们可以做

1. 基于Cortex M0搭建SoC
2. 在SoC系统上进行软件开发

通过搭建SoC，可以很好地学习SoC设计及嵌入式软件编程

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用



硬木课堂
www.emooc.cc

Arm Cortex M3

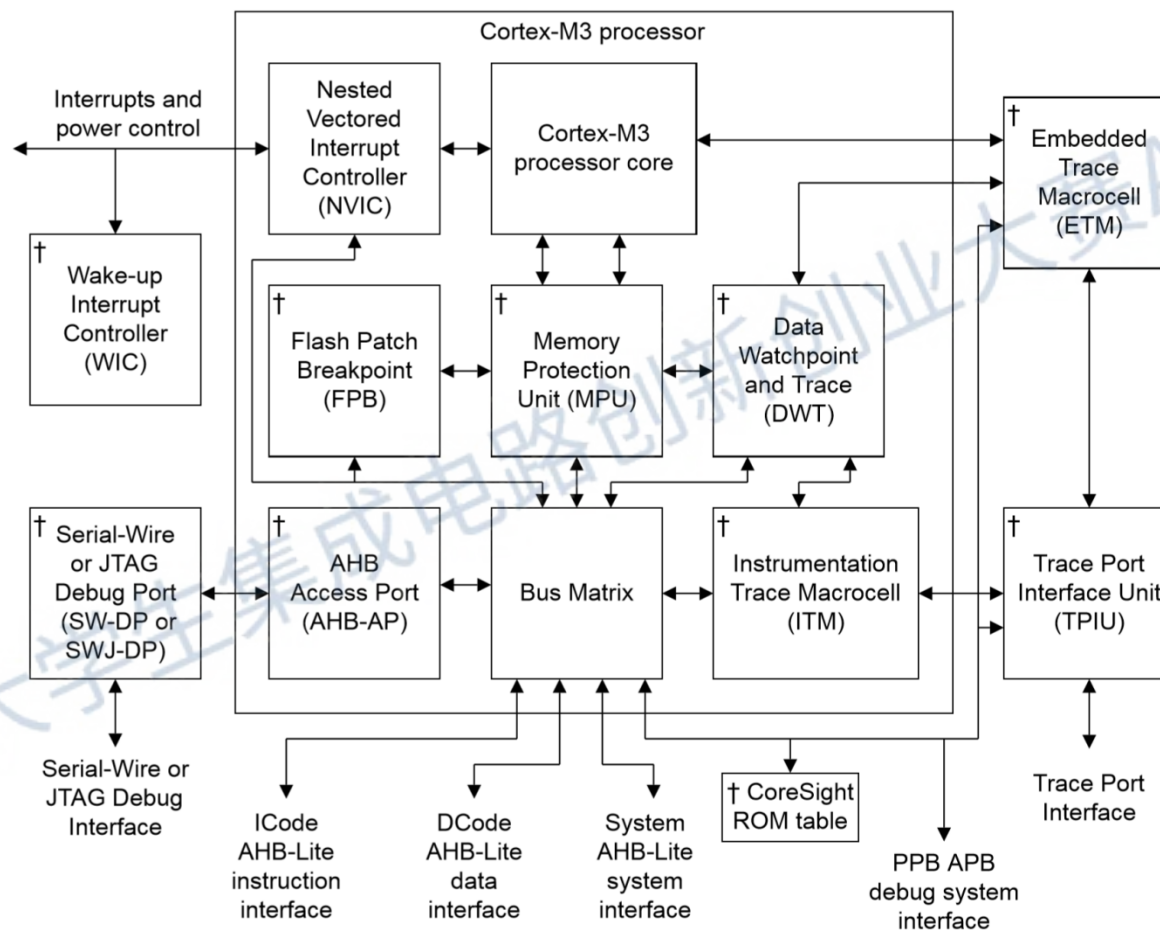
仅限2020年大学生集成电路创新创业大赛Arm杯学习使用

Arm Cortex M3

- 基于Armv7-M架构(参考Armv7-M Architecture Reference Manual)
- 三级流水线
- 使用Thumb/Thumb2 ISA的一个子集
- 32位寻址空间
- 总线接口 AHB-Lite(参考AMBA 3 AHB Lite Protocol Specification)
- 支持NMI以及最大支持240物理中断

仅限2020年大学生集成电路创新创业大赛ARM杯学习使用

Arm Cortex M3 Arch



CPU提供了中断向量端口、AHB-lite端口以及DAP端口

Arm Cortex M3 启动流程

1. 在复位使能时，CPU处于Reset异常状态；
2. 复位释放后，从地址0x00000000出加载栈顶地址，及汇编代码中__Vector的第一行则为栈顶地址；
3. 从地址0x00000004初加载复位处理函数的地址；
4. PC改变为0x00000004中的值，开始执行复位处理，同时CPU的工作状态从异常模式切换为线程模式，开始正常工作。



Arm Cortex M3 中断

CPU中断处理过程：

1. CPU接收到中断信号（IRQ、NMI、Systick等等）
2. 寄存器入栈
3. 根据中断信号查找中断向量表（对应汇编启动代码中的__Vector段），跳转至中断处理函数
4. 中断处理函数执行完成后，寄存器出栈，PC跳转



Arm Cortex M3 中断进入

```
// ExceptionEntry()  
// =====
```

```
ExceptionEntry(integer ExceptionType)
```

```
// NOTE: PushStack() can abandon memory accesses if a fault occurs during the stacking  
//       sequence.
```

```
//       Exception entry is modified according to the behavior of a derived exception,  
//       see DerivedLateArrival() and associated text.
```

```
    PushStack(ExceptionType);
```

```
    ExceptionTaken(ExceptionType);
```

Arm Cortex M3 中断向量表

Table B1-4 Exception numbers

Exception number	Exception
1	Reset
2	NMI
3	HardFault
4	MemManage
5	BusFault
6	UsageFault
7-10	Reserved
11	SVCall
12	DebugMonitor
13	Reserved
14	PendSV
15	SysTick
16	External interrupt 0
.	.
.	.
.	.
16+N	External interrupt N

Table B1-5 Vector table format

Word offset in table	Description, for all pointer address values
0	SP_main. This is the reset value of the Main stack pointer.
Exception Number	Exception using that Exception Number.

Arm Cortex M3 中断入栈

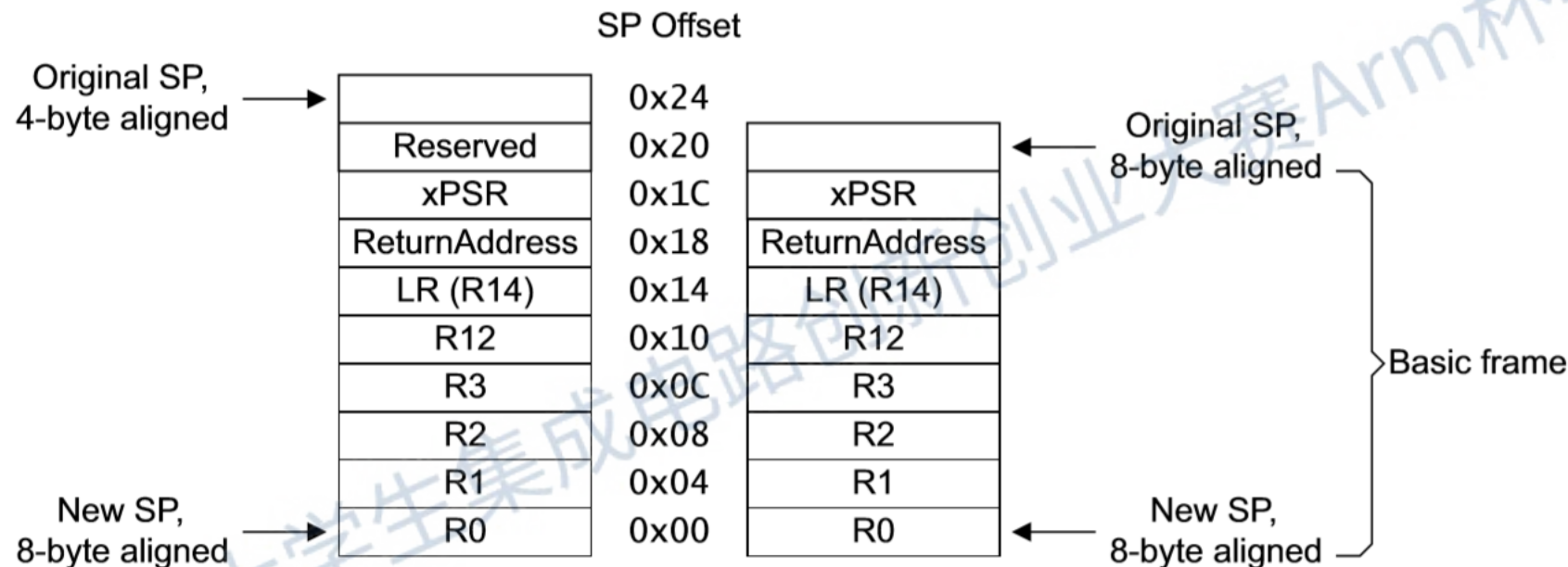


Figure B1-3 Alignment options when stacking the basic frame

Arm Cortex M3 关键信号说明

信号名	描述
HXXXX(以H开头的信号)	总线相关，下一节将会详细介绍
vis_rX_o	通用寄存器，通过仿真观察这些寄存器能够知道相关汇编指令是否正常执行（ADD、MOV等）
vis_msp_o	栈指针
vis_r14_o	链接寄存器，程序返回出错时需要检查此寄存器
vis_pc_o	程序计数器，可以用来判断处理器是否在正常运行
vis_ipsr_o	中断状态寄存器，当发生错误的时候，PSR的值会改变，用于判断发生何种类型错误
LOCKUP	处理器处理NMI或者HardFault时又发生错误后，处理器将会被死锁

以上信号对于理解处理器的运行过程较为重要，在后续实验中需要重视它们随系统运行状态的变化

Arm Cortex M3 端口配置

- 在Arm DesignStart网址下载的Cortex-M3 DesignStart Eval文件资源中找到名为cortexm3ds_logic.v的文件，这便是处理器核的网表形式的Verilog代码。在实验开始前，我们需要对处理器核的时钟、复位、无用端口以及DAP的iobuf进行配置。
- 代码不可读，相关文档对核端口说明很少
- 根据DesignStart资料包中提供的参考设计理解各端口意义

Arm Cortex M3 端口配置

■ PMU端口

```
// PMU  
.ISOLATEN  
.RETAINn
```

```
(1'b1),  
(1'b1),
```

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用



Arm Cortex M3 端口配置

■ RESETS端口

```
// RESETS
```

```
.PORESETn
```

```
.SYSRESETn
```

```
.SYSRESETREQ
```

```
.RSTBYPASS
```

```
.CGBYPASS
```

```
.SE
```

```
(RSTn),  
(cpuresetn),  
(SYSRESETREQ),  
(1'b0),  
(1'b0),  
(1'b0),
```

```
wire SYSRESETREQ;
```

```
reg cpuresetn;
```

```
always @(posedge clk or negedge RSTn)begin
```

```
    if (~RSTn)
```

```
        cpuresetn <= 1'b0;
```

```
    else if (SYSRESETREQ)
```

```
        cpuresetn <= 1'b0;
```

```
    else
```

```
        cpuresetn <= 1'b1;
```

```
end
```



Arm Cortex M3 端口配置

■ SYSTICK端口

```
// SYSTICK
```

```
.STCLK
```

```
.STCALIB
```

```
.AUXFAULT
```

```
(1'b0),
```

```
(26'b0),
```

```
(32'b0),
```

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用



Arm Cortex M3 端口配置

■ SYSTEM CONFIG端口

```
// CONFIG - SYSTEM
```

```
.BIGEND
```

```
.DNOTITRANS
```

```
(1'b0),
```

```
(1'b1),
```

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用



Arm Cortex M3 端口配置

■ DEBUG PORT 端口

```
// SWJDP
```

```
.nTRST
```

```
.SWDITMS
```

```
.SWCLKTCK
```

```
.TDI
```

```
.CDBGPWRUPACK
```

```
.CDBGPWRUPREQ
```

```
.SWDO
```

```
.SWDOEN
```

```
(1'b1),
```

```
(SWDI),
```

```
(swck),
```

```
(1'b0),
```

```
(CDBGPWRUPACK),
```

```
(CDBGPWRUPREQ),
```

```
(SWDO),
```

```
(SWDOEN),
```

```
assign SWDI = SWDIO;
```

```
assign SWDIO = (SWDOEN) ? SWDO : 1'bz;
```

```
wire CDBGPWRUPREQ;
```

```
reg CDBGPWRUPACK;
```

```
always @(posedge clk or negedge RSTn)begin
```

```
    if (~RSTn)
```

```
        CDBGPWRUPACK <= 1'b0;
```

```
    else
```

```
        CDBGPWRUPACK <= CDBGPWRUPREQ;
```

```
end
```


Arm Cortex M3 端口配置

■ IRQ端口

```
// IRQS
.INTISR                (IRQ),
.INTNMI                (1'b0),

wire    [239:0] IRQ;
wire    [3:0]   key_interrupt;
wire    DMA_Event;
wire    uart_interrupt;

assign  IRQ = {234'b0,uart_interrupt,DMA_Event,key_interrupt};
```

Arm Cortex M3 端口配置

■ BUS端口

// I-CODE BUS

```
.HREADYI          (HREADYI),
.HRDATAI          (HRDATAI),
.HRESPi          (HRESPi),
.IFLUSH
.HADDRi
.HTRANSi
.HSIZEi
.HBURSTi
.HPROTi
```

// SYSTEM BUS

```
.HREADYs          (HREADYs),
.HRDATAs          (HRDATAs),
.HRESPs
.EXRESPs
.HADDRs
.HTRANSs
.HSIZEs
.HBURSTs
.HPROTs
.HWDATAs
.HWRITES
```

// D-CODE BUS

```
.HREADYD          (HREADYD),
.HRDATAD          (HRDATAD),
.HRESPD          (HRESPD),
.EXRESPD          (1'b0),
.HADDRD
.HTRANSD          (HTRANSD),
.HSIZED           (HSIZED),
.HBURSTD          (HBURSTD),
.HPROTD           (HPROTD),
.HWDATAD          (HWDATAD),
.HWRITED          (HWRITED),
```



Arm Cortex M3 端口配置

■ OTHERS端口

```
// SLEEP
.RXEV                                (1'b0),
.SLEEPHOLDREQn                      (1'b1),
.SLEEPING                           (SLEEPing),

// EXTERNAL DEBUG REQUEST
.EDBGRQ                             (1'b0),
.DBGRESTART                         (1'b0),

// DAP HMASTER OVERRIDE
.FIXMASTERTYPE                      (1'b0),

// WIC
.WICENREQ                           (1'b0),

// TIMESTAMP INTERFACE
.TSVALUEB                           (48'b0),

// CONFIG - DEBUG
.DBGEN                              (1'b1),
.NIDEN                              (1'b1),
.MPUDISABLE                         (1'b0)
```



硬木课堂
www.emooc.cc

AMBA3 AHB-Lite总线协议

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用

AMBA3 AHB-Lite

AMBA3中的AHB(Advanced High-performance Bus)，高性能总线

1. 可以实现高性能的同步设计
2. 支持多个总线主设备(参考《Multi-layer AHB Overview》)
3. 提供高带宽操作

AHB-Lite是AHB的子集，简化了AHB总线的设计，只有一个主设备



AHB-Lite概述

对于AHB-Lite，包含数据总线、地址总线和额外的控制信号

1. 数据总线用于交换数据信息
2. 地址总线用于选择一个外设，或者一个外设中的某个寄存器
3. 控制信号用于同步和识别tradeoff

仅限2020年大学生集成电路创新创业大赛AIH杯学习使用

AHB-Lite信号说明

名称	来源	描述
HADDR[31:0]	Master	传输地址
HBURST[2:0]	Master	Burst类型
HSIZE[2:0]	Master	数据宽度 00: 8bit Byte 01: 16bit Halfword 10: 32bit Word
HTRANS[1:0]	Master	传输类型 00: IDLE, 无操作 01: BUSY 10: NONSEQ, 主要的传输方式 11: SEQ
HWDATA[31:0]	Master	核发出的写数据
HWRITE	Master	读写选择 (1: 写, 0: 读)
HRDATA[31:0]	Slave	外设返回的读数据
HREADOUT	Slave	何时传输完成 (通常为1)
HRESP	Slave	传输是否成功 (通常为0)



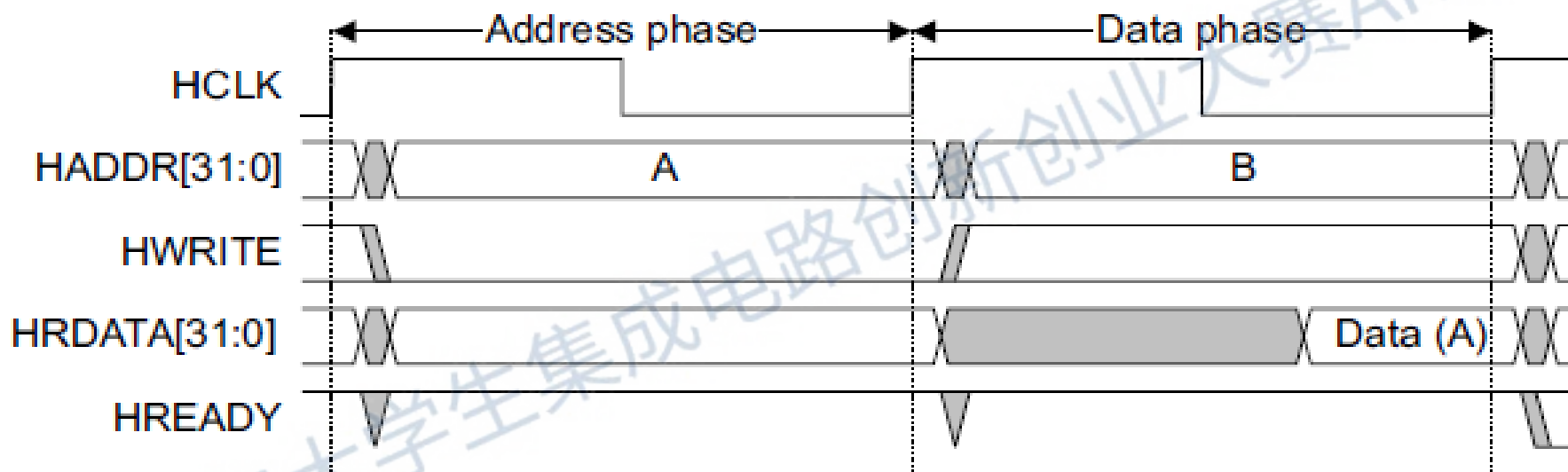
AHB-Lite传输类型

HTRANS[1:0]	Master	传输类型 00: IDLE, 无操作 01: BUSY 10: NONSEQ, 主要的传输方式 11: SEQ
-------------	--------	---

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用

AHB-Lite NONSEQ时序

■ 基本读操作



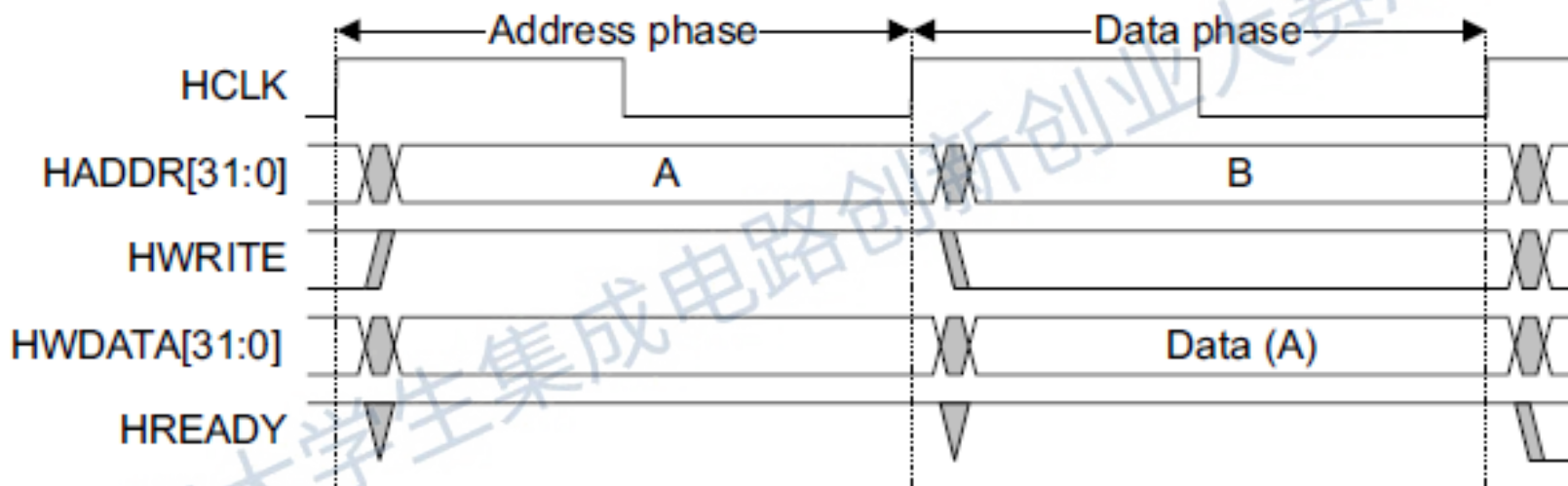
Master需要从外设读取数据时，总共需要经历两个阶段：Address phase & Data phase

Address phase：Master把地址输出在地址总线上，直到HREADY为1后进入Data phase

Data phase：Master会在HREADY为1时读取数据总线HRDATA上的数据，至此传输完成

AHB-Lite NONSEQ时序

■ 基本写操作



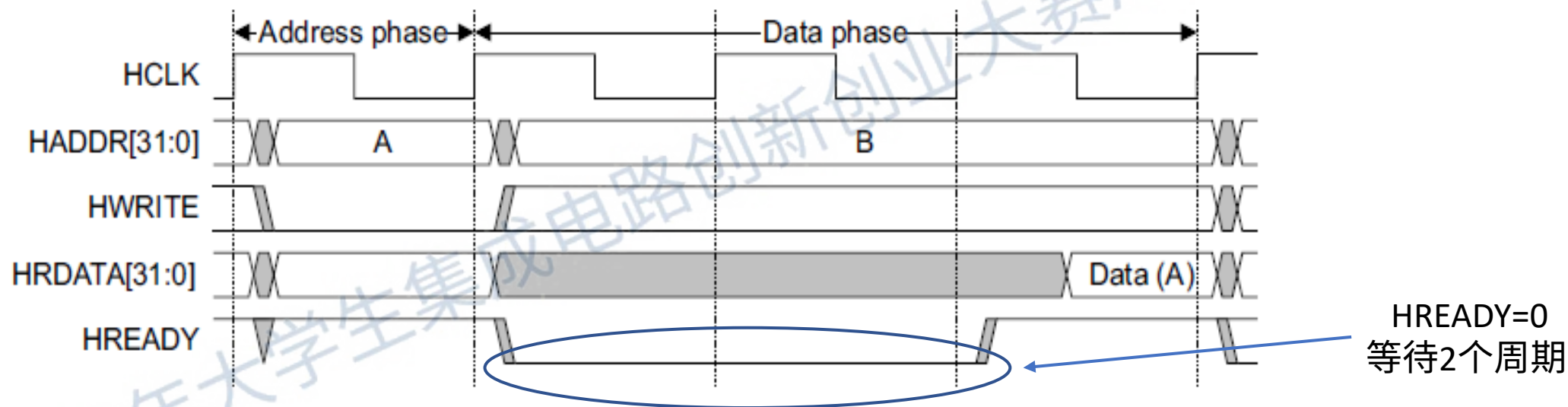
Master需要从外设读取数据时，总共需要经历两个阶段：Address phase & Data phase

Address phase：Master把地址输出在地址总线上，直到HREADY为1后进入Data phase

Data phase：Master把写数据放在数据总线HWDATA上，直到HREADY为1时传输完成

AHB-Lite NONSEQ时序

■ 具有等待的读操作



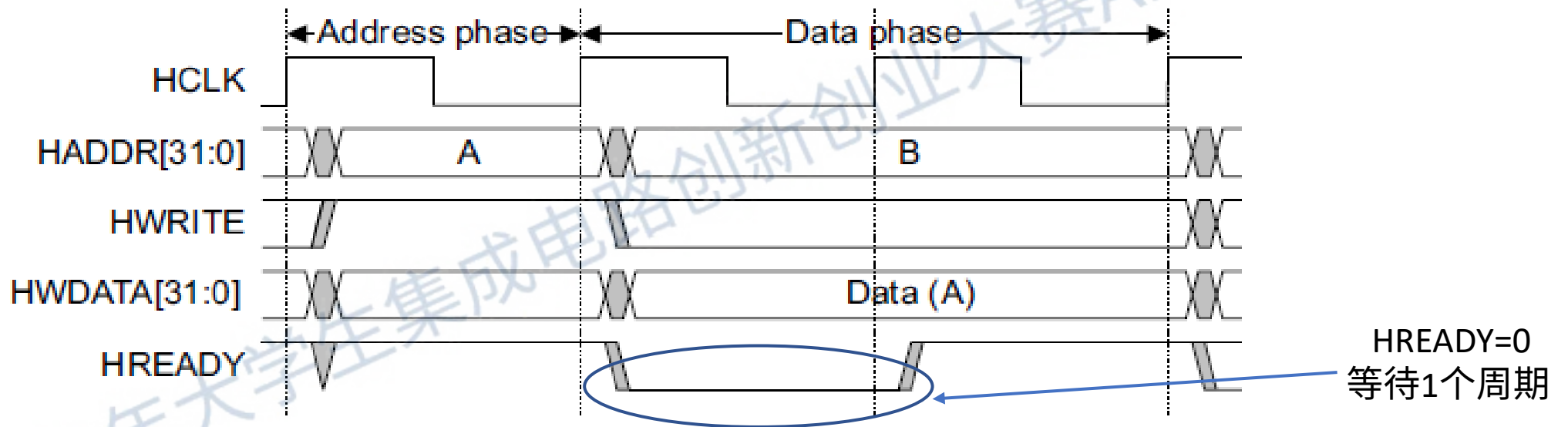
Master需要从外设读取数据时，总共需要经历两个阶段：Address phase & Data phase

Address phase：Master把地址输出在地址总线上，直到HREADY为1后进入Data phase

Data phase：Master会在HREADY为1时读取数据总线HRDATA上的数据，至此传输完成

AHB-Lite NONSEQ时序

■ 具有等待的写操作



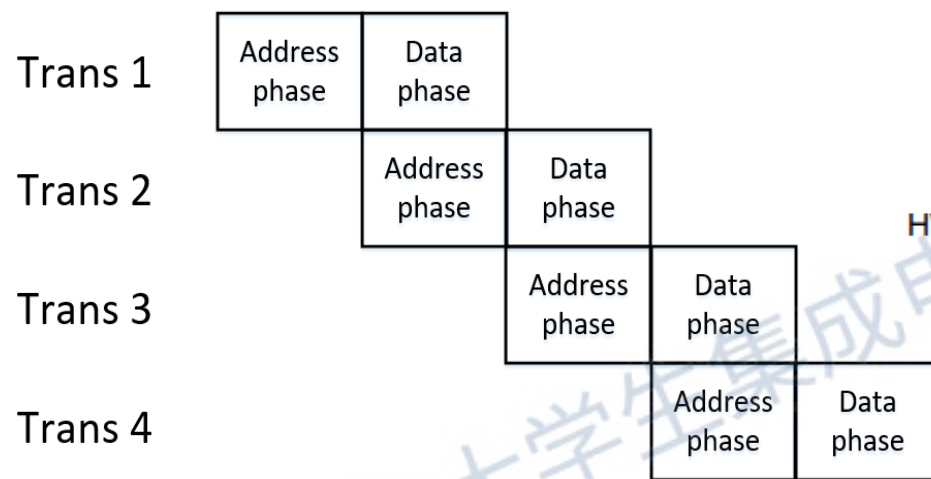
Master需要从外设读取数据时，总共需要经历两个阶段：Address phase & Data phase

Address phase：Master把地址输出在地址总线上，直到HREADY为1后进入Data phase

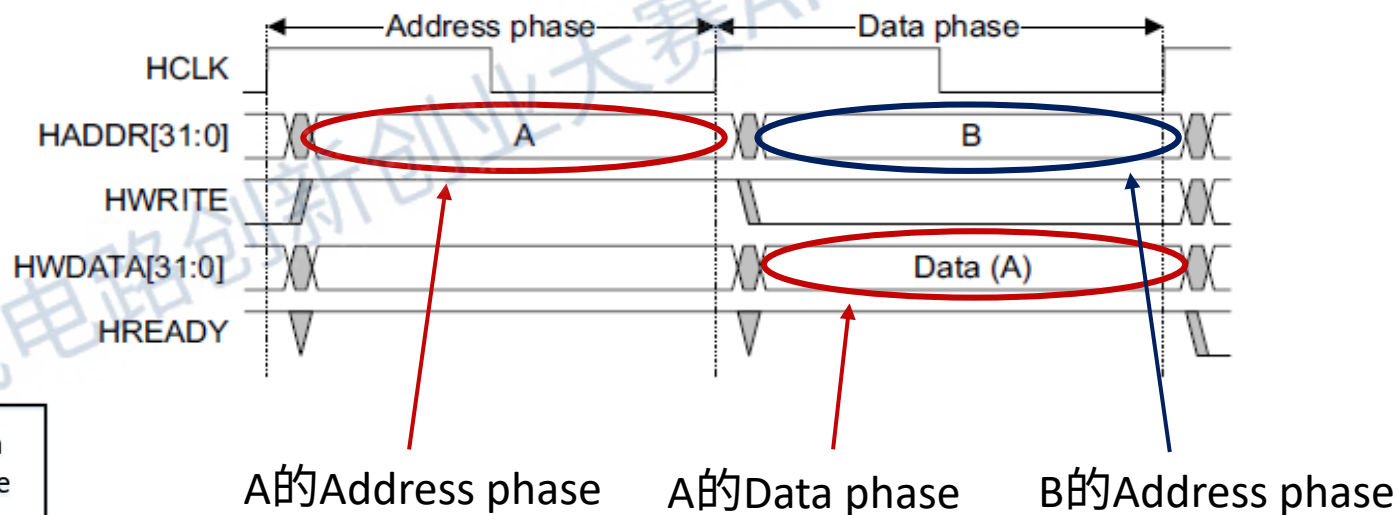
Data phase：Master把写数据放在数据总线HWDATA上，直到HREADY为1时传输完成

AHB-Lite 流水线传输

■ 总线流水



基本写操作时序



在基本读写操作中，把A与B看作两次传输，A的data phase同时也作为B的地址phase
实现上图所示的流水线传输

AHB-Lite Single Master Arch

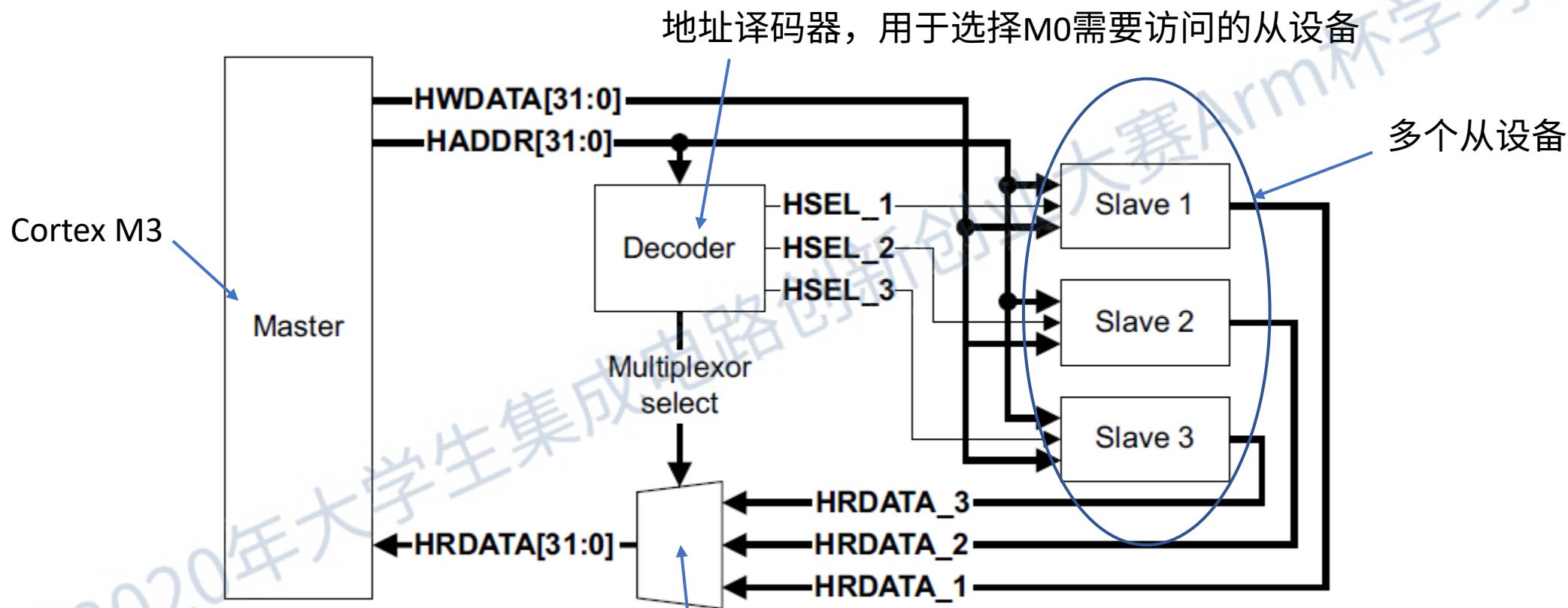
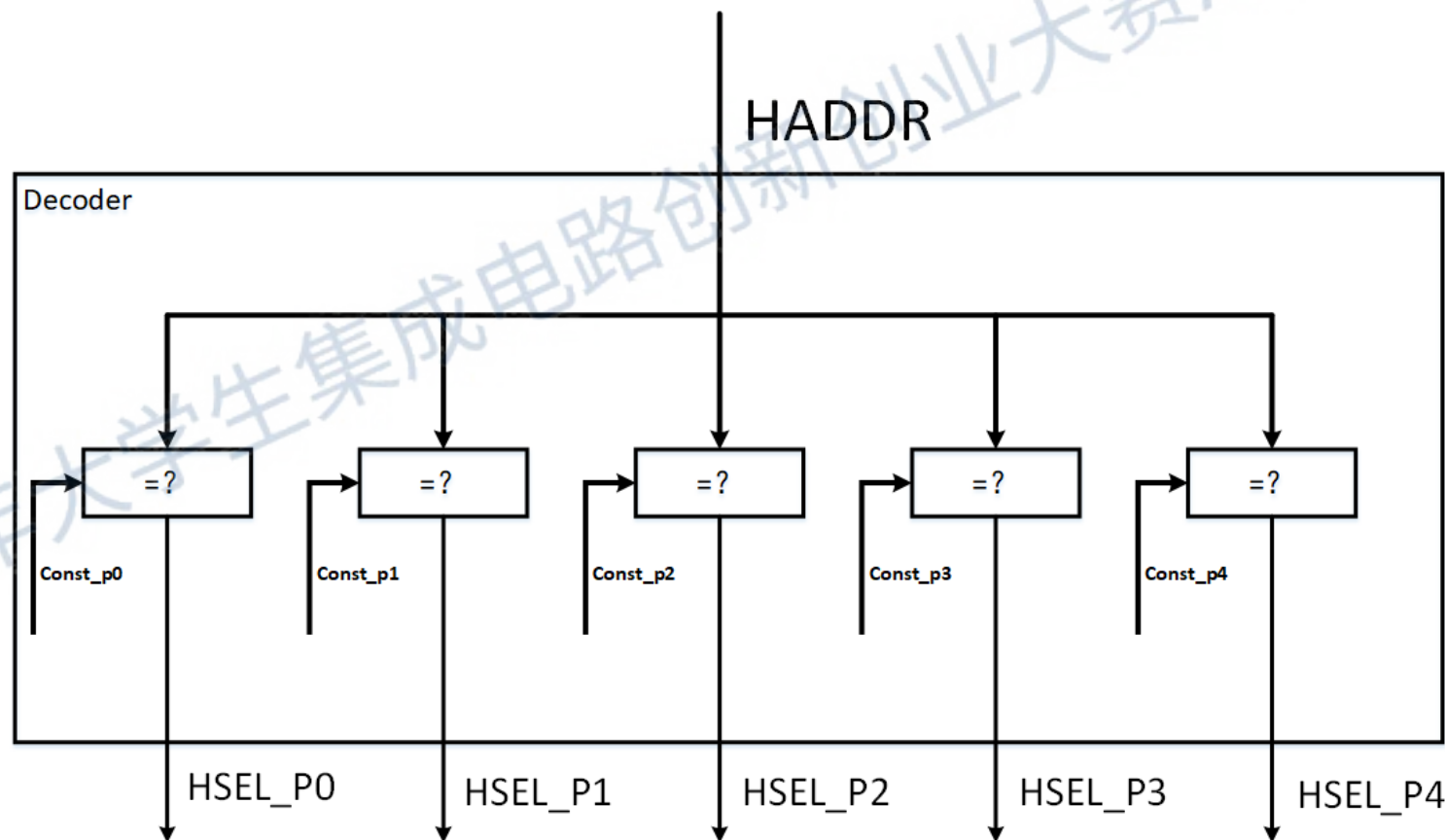


Figure 1-1 AHB-Lite block diagram

从设备多路复用器，用于从多个从设备中选择所要读取的数据和响应信号

AHB-Lite Single Master Decoder

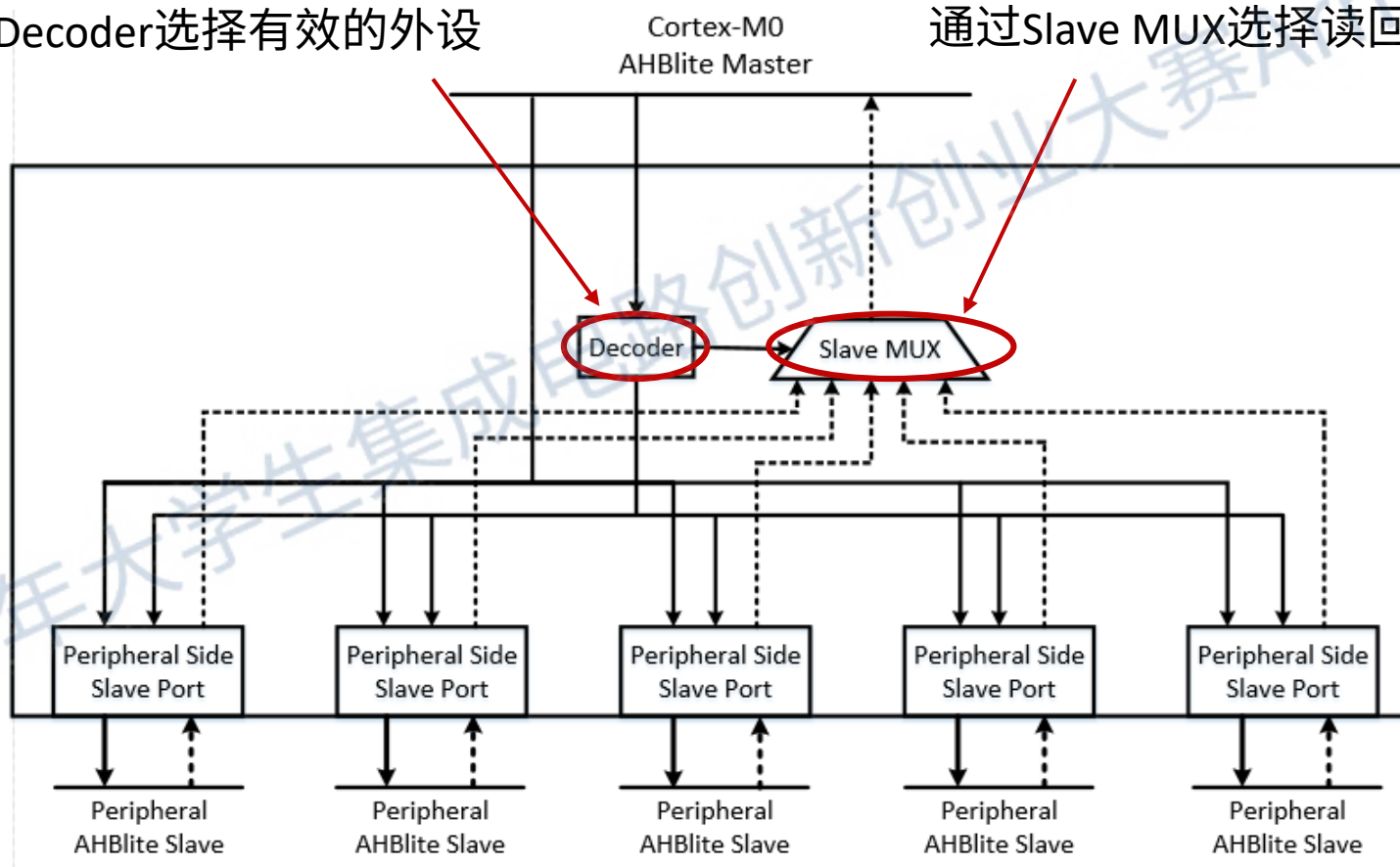
对地址译码，输出外设选择信号HSEL，选择特定外设



AHB-Lite Single Master Interconnect

通过Decoder选择有效的外设

通过Slave MUX选择读回的数据

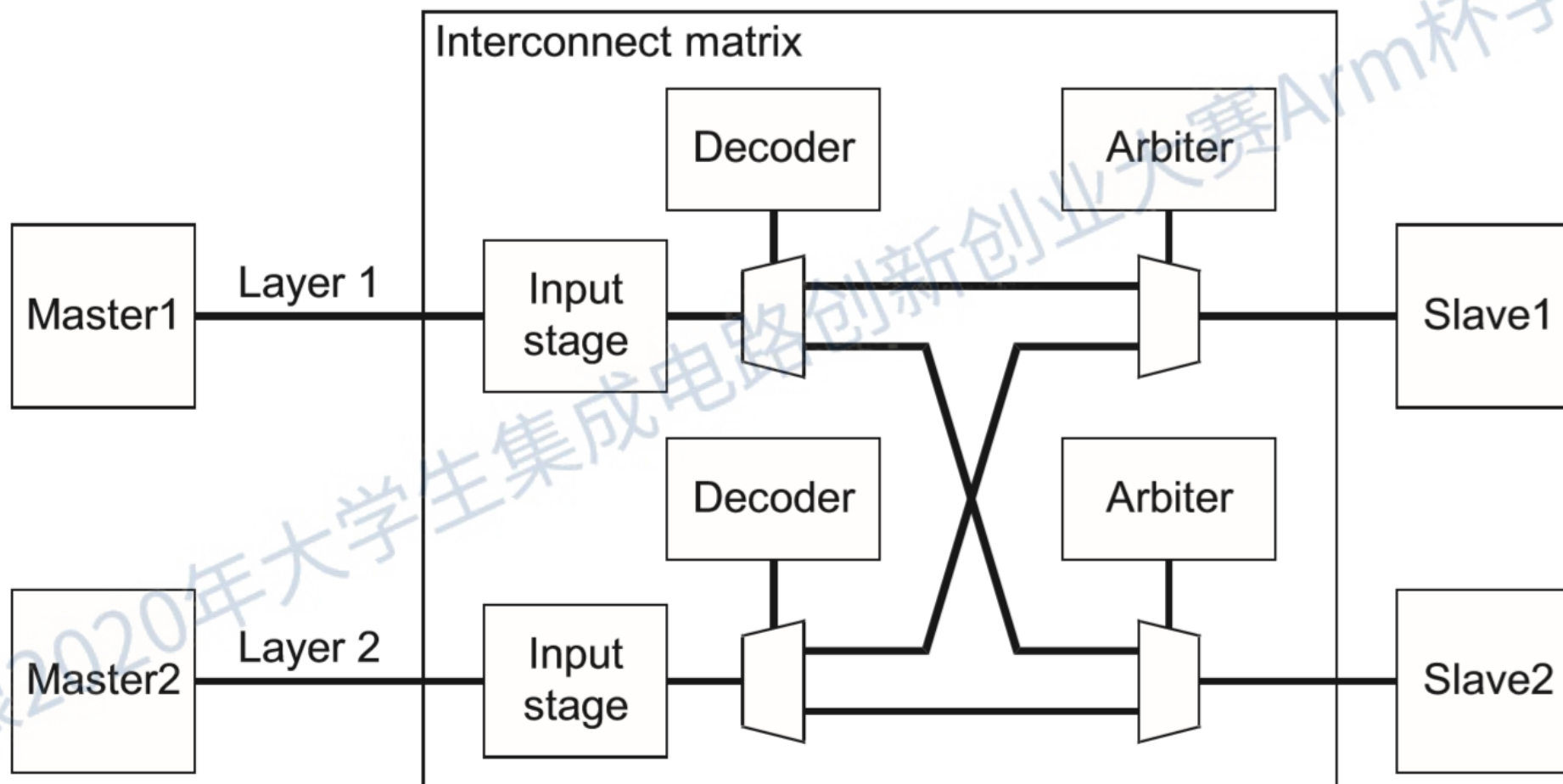


扩展外设步骤—Single Master

1. 扩展Decoder
2. 在Interconnect中增加Slave Port
3. 在Interconnect中将新增的Slave Port与扩展的Decoder HSEL信号以及Slave MUX连接
4. 顶层连接外设、外设接口、总线
5. C语言程序中编写外设接口结构体

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用

AHB-Lite Multi Master Arch





AHB-Lite BUS MTX Generate by CMSDK

```
my.xml
C:\Users\tianj> OneDrive\Study\Cortex_M\Cortex_M3_Eval\cmsdk\logical\cmsdk_ahb_busmatrix> xml> my.xml

64
65 <slave_interface name="S2">
66   <sparse_connect interface="M1"/>
67   <sparse_connect interface="M2"/>
68   <sparse_connect interface="M3"/>
69   <sparse_connect interface="M4"/>
70   <address_region interface="M1" mem_lo="20000000" mem_hi="20007fff" remap
71   <address_region interface="M2" mem_lo="40000000" mem_hi="4000ffff" remap
72   <address_region interface="M3" mem_lo="40010000" mem_hi="4001ffff" remap
73   <address_region interface="M4" mem_lo="40030000" mem_hi="4003ffff" remap
74 </slave_interface>
75
76
77 <slave_interface name="S3">
78   <sparse_connect interface="M1"/>
79   <sparse_connect interface="M3"/>
80   <sparse_connect interface="M4"/>
81   <address_region interface="M1" mem_lo="20000000" mem_hi="20007fff" remap
82   <address_region interface="M3" mem_lo="40010000" mem_hi="4001ffff" remap
83   <address_region interface="M4" mem_lo="40030000" mem_hi="4003ffff" remap
84 </slave_interface>
85
86 <!-- Master interface definitions -->
87
88 <master_interface name="M0"/>
89 <master_interface name="M1"/>
90 <master_interface name="M2"/>
91 <master_interface name="M3"/>
92 <master_interface name="M4"/>
93
94 <!-- - - - - - *** DO NOT MODIFY BELOW THIS LINE *** - - - - -
95
96 </cfgfile>
97
```

```
sudo bin/BuildBusMatrix.pl -xml_dir xml -cfg my.xml -over -verbose
```

Creating the bus matrix variant...

```
- Rendering 'L1MTXOutStgM1.v'
- Rendering 'L1MTX.v'
- Rendering 'L1MTXDecS2.v'
- Rendering 'L1MTXOutStgM2.v'
- Rendering 'L1MTXDecS1.v'
- Rendering 'L1MTXOutStgM0.v'
- Rendering 'L1MTXArbM2.v'
- Rendering 'L1MTXArbM0.v'
- Rendering 'L1MTXArbM4.v'
- Rendering 'L1MTXOutStgM3.v'
- Rendering 'L1MTX_lite.v'
- Rendering 'L1MTXDecS3.v'
- Rendering 'L1MTXInStg.v'
- Rendering 'L1MTXOutStgM4.v'
- Rendering 'L1MTXDecS0.v'
- Rendering 'L1MTX_default_slave.v'
- Rendering 'L1MTXArbM1.v'
- Rendering 'L1MTXArbM3.v'
```

Done!

Used the following parameters:

```
ration file      : 'xml/my.xml'
el name         : 'L1MTX'
nterfaces       : 4
interfaces      : 5
cture type      : 'ahb2'
tion scheme     : 'round'
map             : user defined
ivity mapping    : S0 -> M0,
                  S1 -> M0,
                  S2 -> M1, M2, M3, M4,
                  S3 -> M1, M3, M4
ivity type      : sparse
data width      : 32
address width   : 32
gnal width      : 32
ration directory : './verilog/built'
directory       : './verilog/src'
te mode         : enabled
```

扩展外设步骤—Multi Master

1. 在XML中扩展Master Port
2. 执行脚本生产新的总线矩阵
3. 顶层连接外设、外设接口、总线
4. C语言程序中编写外设接口结构体

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用



硬木课堂
www.emooc.cc

Arm 杯破题与赛程规划

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用



赛题分析

- 1. 选题**内容不限**，针对**行为检测、产品外观缺陷分析、智能语音交互、人体健康状况智能监控、智能信号分析**等应用合理选题，**鼓励跨学科融合**，SoC本身应具备**较强的智能处理能力**，**不能**将智能算法的**主体部分**部署在**云端或其它高性能计算设备上**。
- 2. 使用**ArmCortex-M3 DesignStart Eval**提供的处理器IP，在指定的FPGA平台上构建**简单的Cortex-M3片上系统**。系统应至少包含：
 - 1) ArmCortex-M3 DesignStart处理器；
 - 2) 利用片上或板上资源实现的ROM与RAM；
 - 3) 与芯片外部引脚连接的GPIO外设。
 - 使用KeilμVision工具编写并生成软件程序，实现**GPIO输出引脚跟随GPIO输入引脚变化**。将对应的输入、输出引脚连接至板上开关与LED，**确认程序正确运行**。
- 3. 在可编程逻辑器件平台上**利用板载资源或扩展的硬件资源**，为Arm SoC添加**信号的预处理、采集、人机交互等接口功能**；
- 4. 为Arm SoC添加**具备执行人工智能/机器学习等智能处理算法的硬件加速器**，具体集成形式可采用**协处理器/独立加速器/通道加速器**等多种形式，在设计中应突出**硬件加速器对系统的优化效果**
- 5. 以**软硬协同**的思想对SoC进行**全面优化**，确定合理的软硬件任务划分，**分析优化前后SoC整体性能的变化**。

提交文件

■ 8. 设计数据:

- 1) 系统原理图;
- 2) 软硬件代码;
- 3) 仿真和测试结果;
- 4) 现场答辩和演示
- 5) 系统设计方案
- 6) 软硬件任务划分
- 7) 加速器设计细节
- 8) 仿真图等验证结果
- 9) 现场演示智能SoC功能

其它注意事项

■ 其他注意事项：

- 1) 参赛所选用的FPGA开发平台限定于Xilinx Spartan 6/7或Artix-7系列，Intel Cyclone IV / V E / 10系列，不得采用内含硬核处理器的FPGA芯片（包括但不限于ZYNQ，Cyclone V SE等）。具体型号和开发板厂家不限。
- 2) Arm携手合作伙伴将为参赛队提供免费的开发板借用服务（需要押金），在报名结束后开通借用通道。因为开发板数量有限（预计Artix-7版100片，Cyclone-V版50片），申请者需在报名时提交一份开发板申请书，简述项目开发思路和以往成果，择优发放。

板卡介绍——Altera版

FPGA + Arm Cortex-M SoC系统实验套件

Altera进阶版 5CEFA5/5CEFA7

IO组1, 共22条IO

- 并口LCD屏接口
- 或11对LVDS差分对

IO组2, 共18条IO:

- 9对LVDS或18条CMOS

IO组3, 共16条IO:

- CMOS摄像头接口
- 或8对LVDS

数码管和蜂鸣器:

- 6位数码管
- 无源蜂鸣器

高速ADC和DAC

- Maxim模拟前端
- 双通道8位22MSPS ADC
- 双通道10位22MSPS DAC
- 模拟信号调理前端

音频Codec和SD卡:

- 24位立体声输入输出
- 1W 功率输出
- SD卡插槽 (背面)

电平开关和LED

- 12位电平开关
- 12位LED
- 为数字电路课程服务

千兆以太网

DC 5V辅助供电

VGA

USB Blaster

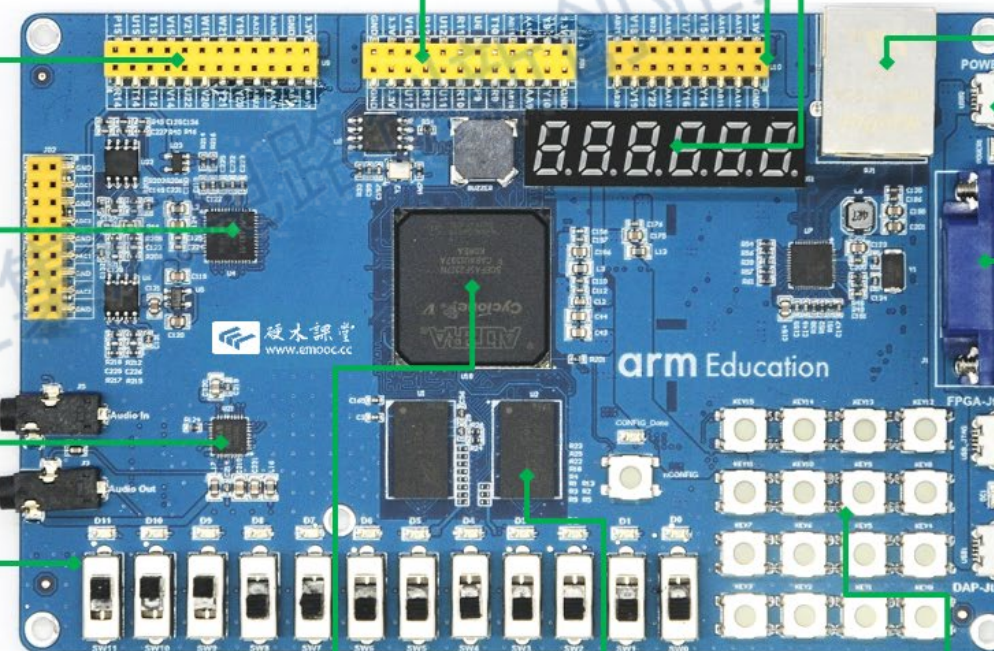
CMSIS-DAP

5CEFA5
CEFA7

32位DDR3L

- 667Mb/s
- 128MByte

4 x 4矩阵键盘





板卡介绍——Xilinx版

FPGA + Arm Cortex-M SoC系统实验套件

Xilinx进阶版 XC7A75T/200T

IO组1, 共22条IO

- 并口LCD屏接口
- 或10对LVDS差分对+2IO

IO组2, 共18条IO:

- 8对差分XADC输入+2IO
- 或18条CMOS或9对LVDS

IO组3, 共16条IO:

- CMOS摄像头接口
- 或8对LVDS

数码管和蜂鸣器:

- 6位数码管
- 无源蜂鸣器

高速ADC和DAC

- Maxim模拟前端
- 双通道8位22MSPS ADC
- 双通道8位22MSPS DAC
- 模拟信号调理前端

音频Codec和SD卡:

- 24位立体声输入输出
- 1W 功率输出
- SD卡插槽 (背面)

电平开关和LED

- 12位电平开关
- 12位LED
- 为数字电路课程服务

千兆以太网

DC 5V辅助供电

VGA

FPGA下载口

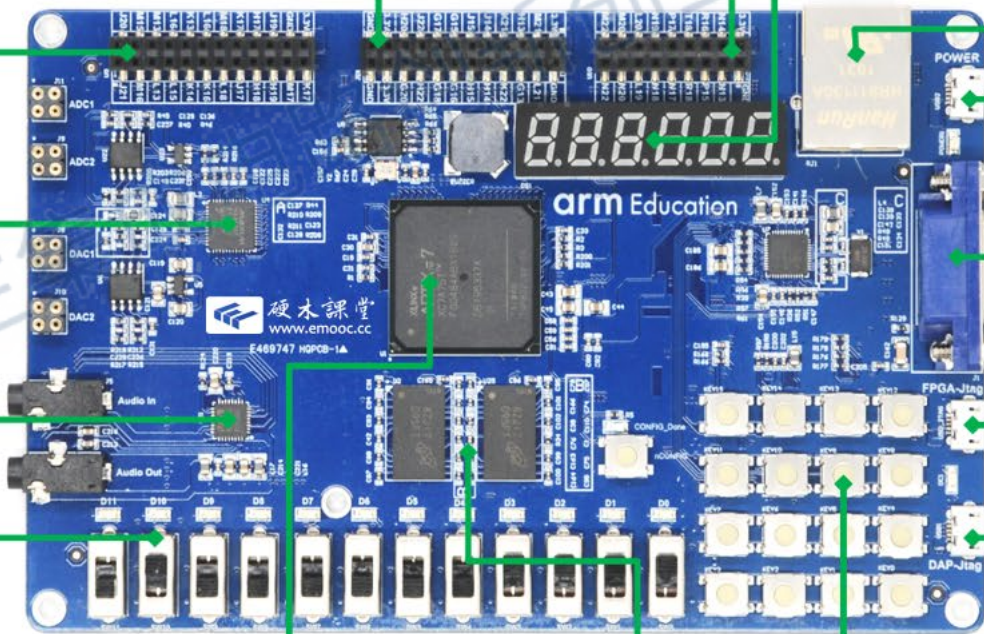
CMSIS-DAP

32位DDR3L

- 800Mb/s
- 256MByte

4 x 4矩阵键盘

XC7A75T/200T





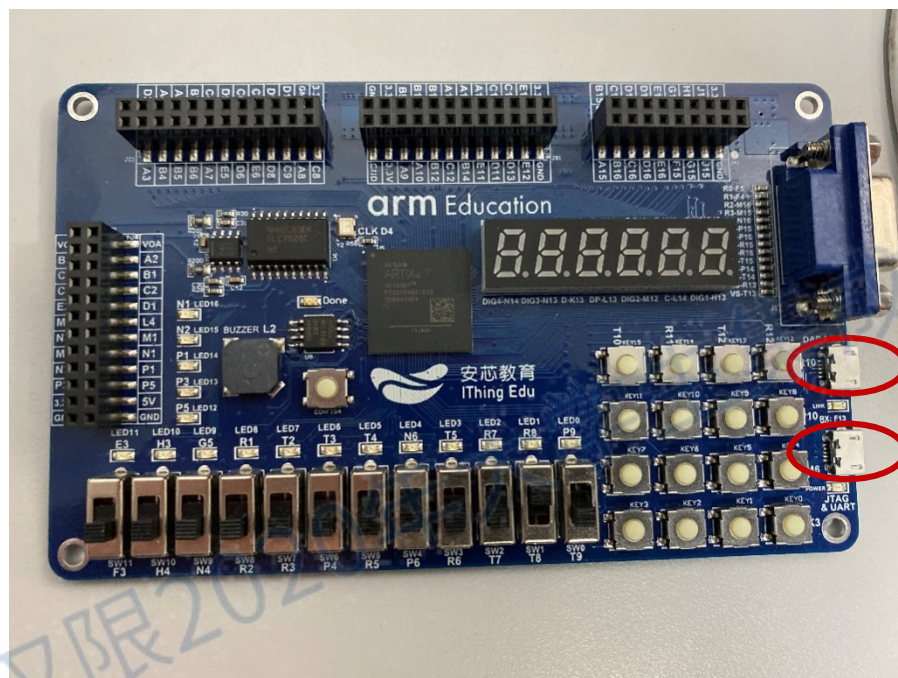
硬木课堂
www.emooc.cc

后续培训安排

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用

实验准备

■ 硬件平台



Xilinx Arty7 75t

■ 软件



ModelSim - Intel FPGA Starter Edition
10.5b (Quartus Prime 18.1)

DAP-LINK调试接口

FPGA配置接口



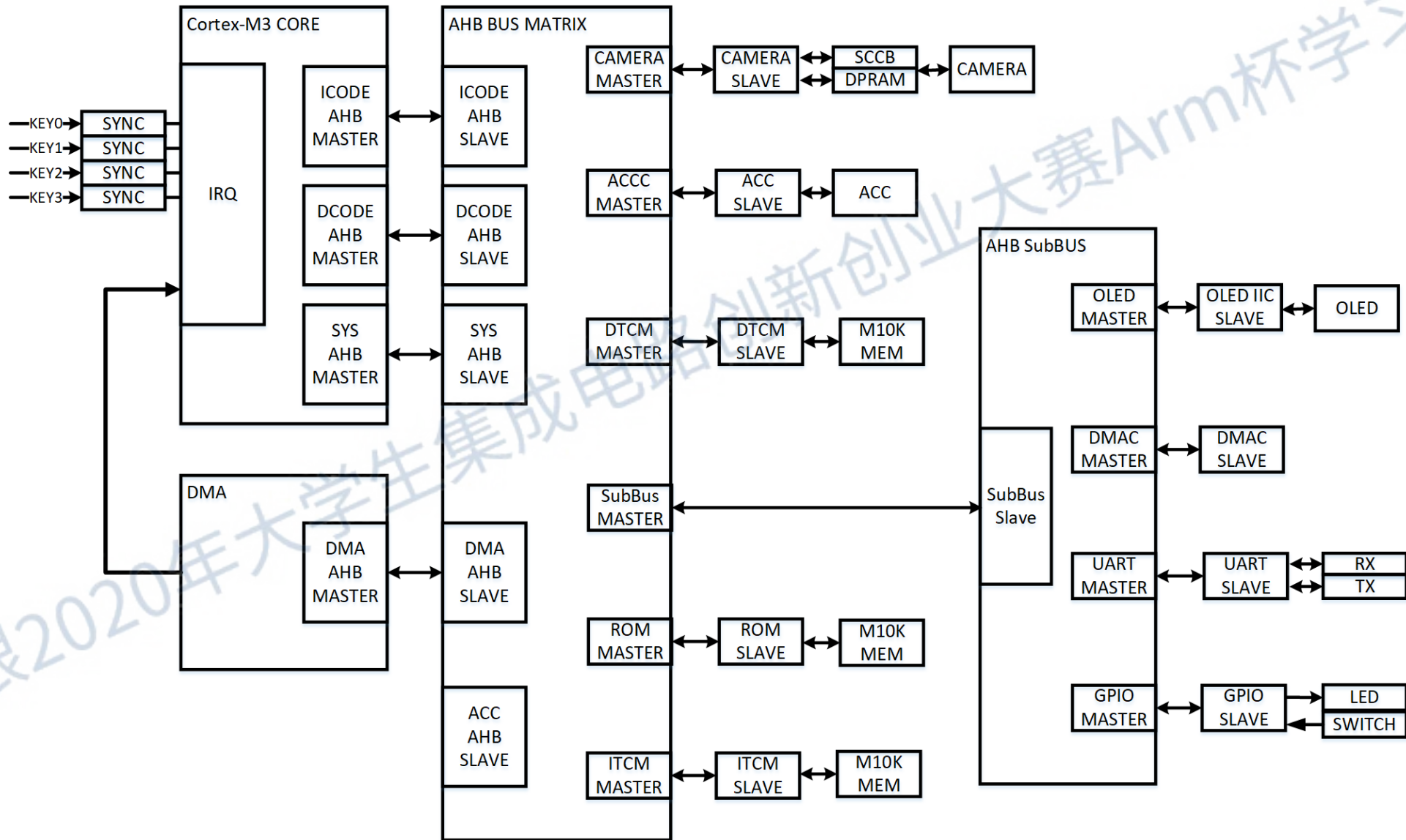
Vivado 2018.3



Keil uVision5

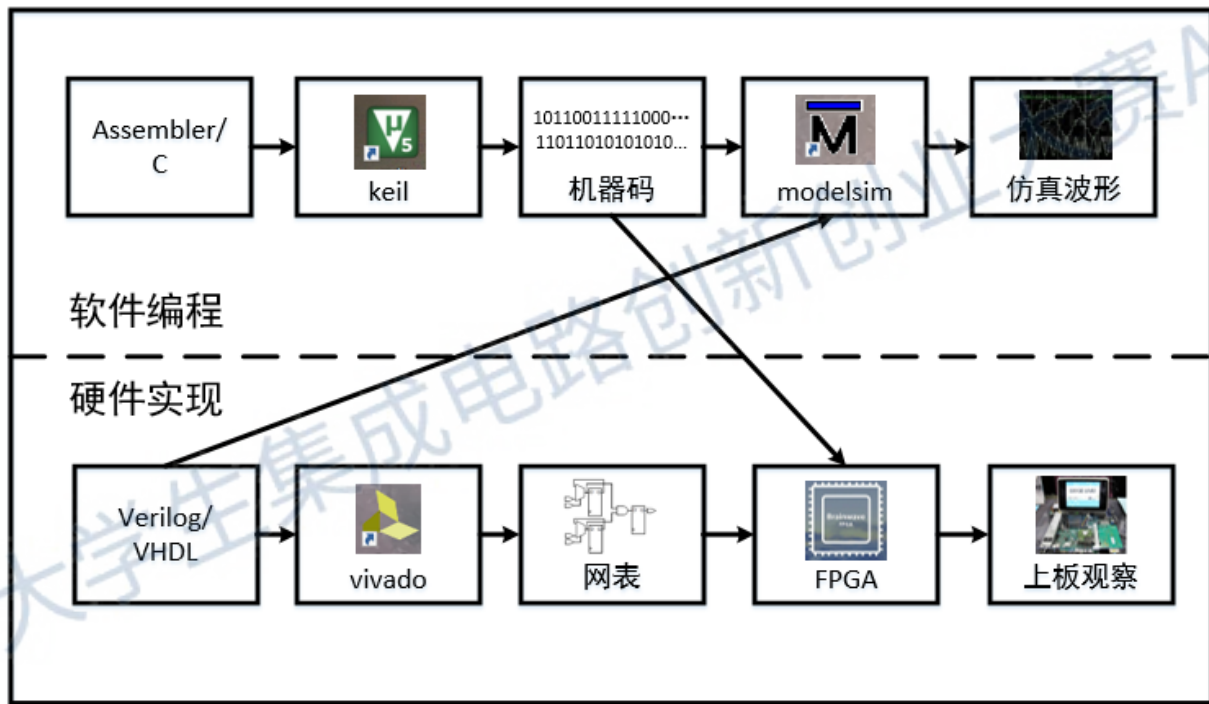


SoC示例





设计流程

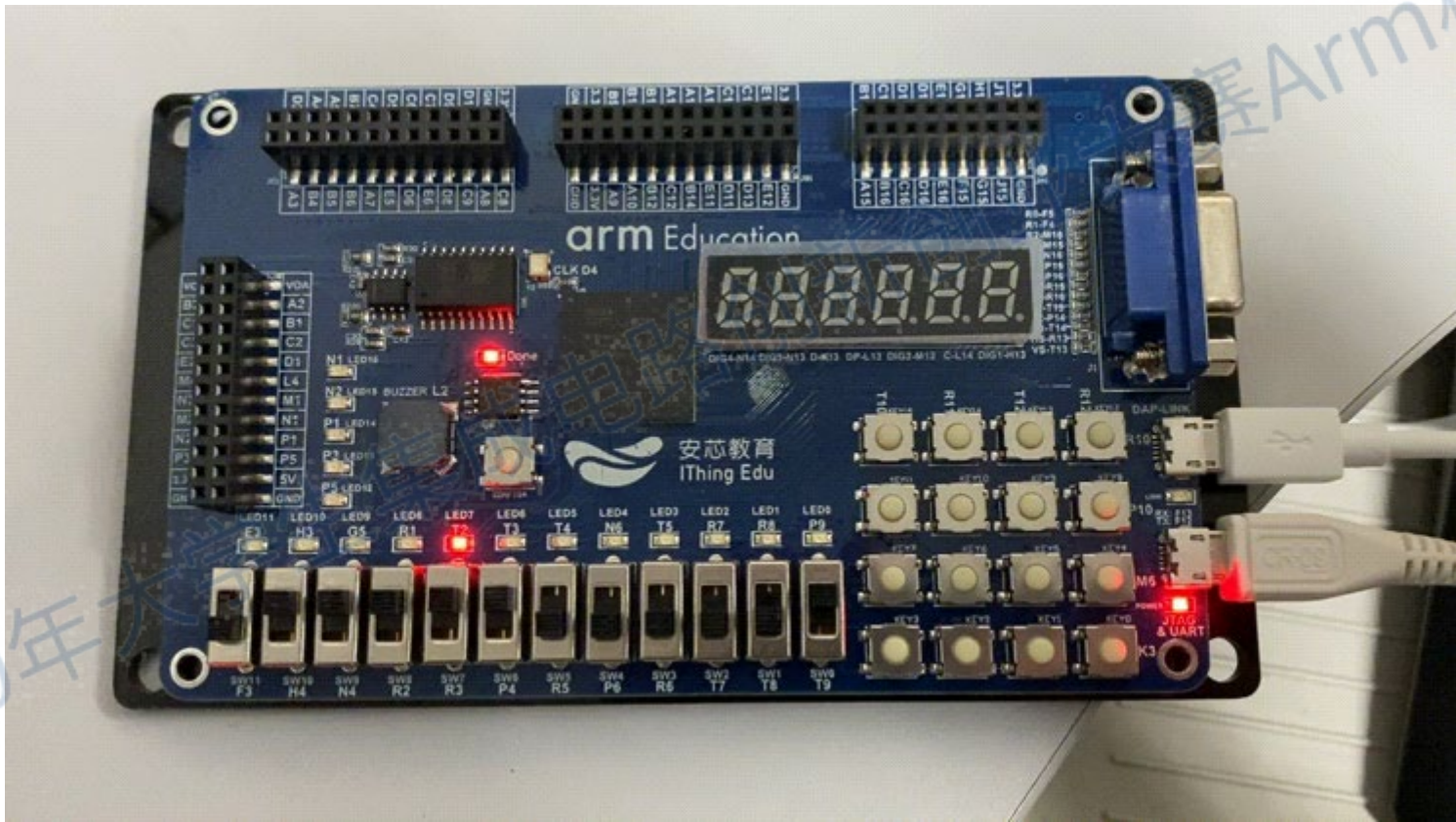


软硬件关系



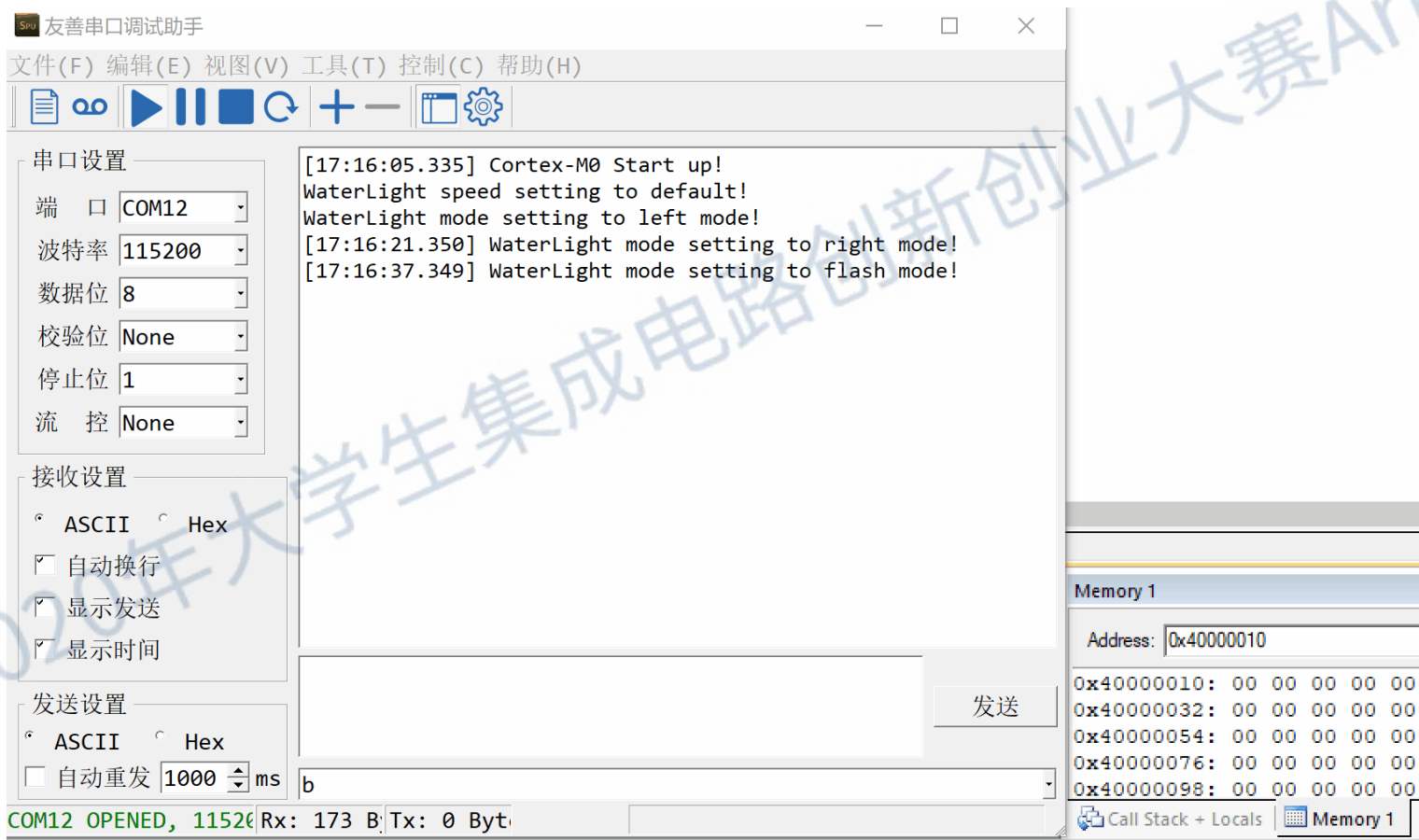
硬木课堂
www.emoooc.cc

实验之流水灯



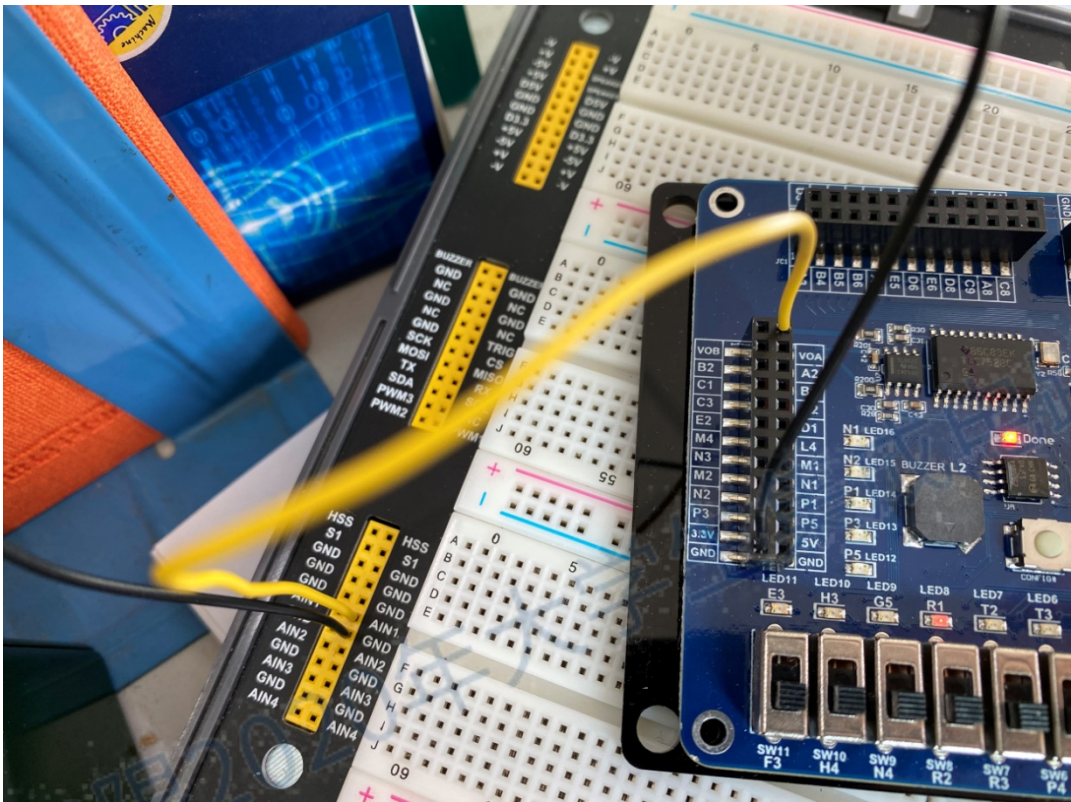
实验之UART

发送数据时调试窗口变化

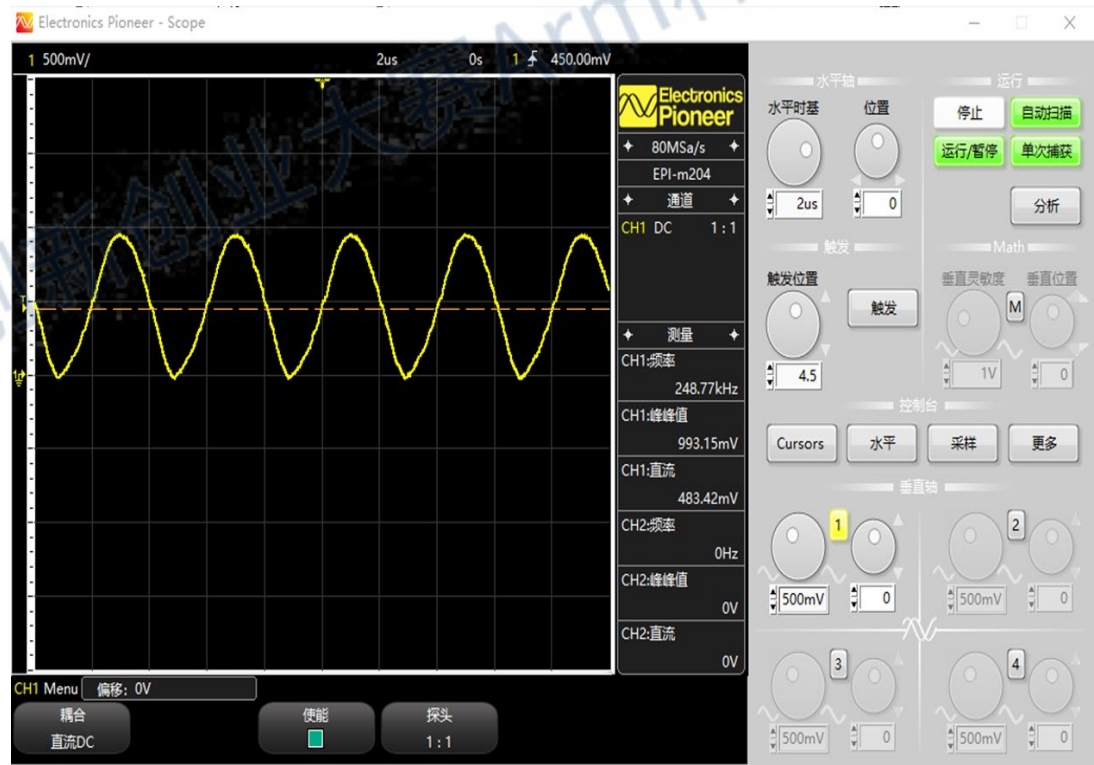


61 62 63分别为
a b c的ASCII码
(97 98 99)
十六进制表示

实验之虚拟仪器



连接方式



观察到的波形



实验之LCD屏幕驱动



Coming Soon...

- 教材/实验指导书
- 在线培训
- 培训/演示视频
- 外部接口模块案例
- SoC软件驱动例程

欢迎到极术社区提问和讨论



Jishu AIoT
developer community
极术社区



<https://aijishu.com/questions>

Thank you!

仅限2020年大学生集成电路创新创业大赛Arm杯学习使用